

AFRILANG Stdlib

std/c/graphmatch

Bibliothèque AFRILANG `std/c/graphmatch` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
maxMatching, m

```
`import "std/c/graphmatch" · `use graphmatch`
```

- `maxMatching(adj list number, n number) ? number`
- `matchingPairs(adj list number, n number) ? list number`
- `isPerfectMatching(adj list number, n number) ? bool`
- `unmatchedCount(adj list number, n number) ? number`
- `matchingWeight(adj list number, n number) ? number`
- `vertexCoverSize(adj list number, n number) ? number`
- `isMatched(adj list number, n number, node number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphmatch` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : maxMatching, m

```
`import "std/c/graphmatch"` . `use graphmatch`  
- `maxMatching(adj list number, n number) ? number`  
- `matchingPairs(adj list number, n number) ? list number`  
- `isPerfectMatching(adj list number, n number) ? bool`  
- `unmatchedCount(adj list number, n number) ? number`  
- `matchingWeight(adj list number, n number) ? number`  
- `vertexCoverSize(adj list number, n number) ? number`  
- `isMatched(adj list number, n number, node number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphmatch"  
use graphmatch
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? maxMatching(adj list number, n number) ? number  
? matchingPairs(adj list number, n number) ? list  
? isPerfectMatching(adj list number, n number) ? bool  
? unmatchedCount(adj list number, n number) ? number  
? matchingWeight(adj list number, n number) ? number  
? vertexCoverSize(adj list number, n number) ? number  
? isMatched(adj list number, n number, node number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphmatch"  
use graphmatch  
  
create result = maxMatching(/* args */)   
say result  
  
create result = matchingPairs(/* args */)   
say result  
  
create result = isPerfectMatching(/* args */)   
say result  
  
create result = unmatchedCount(/* args */)   
say result  
  
create result = matchingWeight(/* args */)   
say result  
  
create result = vertexCoverSize(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/graphmatch"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module graphmatch
```

```
function maxMatching(adj list number, n number) returns number
end

function matchingPairs(adj list number, n number) returns list number
end

function isPerfectMatching(adj list number, n number) returns bool
end

function unmatchedCount(adj list number, n number) returns number
end

function matchingWeight(adj list number, n number) returns number
end

function vertexCoverSize(adj list number, n number) returns number
end

function isMatched(adj list number, n number, node number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphmatch` (tier complex, category complex). Exports 7 function(s):
maxMatching, matchingPairs,

```
`import "std/c/graphmatch" . `use graphmatch`  
- `maxMatching(adj list number, n number) ? number`  
- `matchingPairs(adj list number, n number) ? list number`  
- `isPerfectMatching(adj list number, n number) ? bool`  
- `unmatchedCount(adj list number, n number) ? number`  
- `matchingWeight(adj list number, n number) ? number`  
- `vertexCoverSize(adj list number, n number) ? number`  
- `isMatched(adj list number, n number, node number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphmatch"  
use graphmatch
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? maxMatching(adj list number, n number) ? number  
? matchingPairs(adj list number, n number) ? list  
? isPerfectMatching(adj list number, n number) ? bool  
? unmatchedCount(adj list number, n number) ? number  
? matchingWeight(adj list number, n number) ? number  
? vertexCoverSize(adj list number, n number) ? number  
? isMatched(adj list number, n number, node number) ? bool
```

4. Usage examples

```
import "std/c/graphmatch"  
use graphmatch  
  
create result = maxMatching(/* args */)   
say result  
  
create result = matchingPairs(/* args */)   
say result  
  
create result = isPerfectMatching(/* args */)   
say result  
  
create result = unmatchedCount(/* args */)   
say result  
  
create result = matchingWeight(/* args */)   
say result  
  
create result = vertexCoverSize(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphmatch"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphmatch
```

```
function maxMatching(adj list number, n number) returns number
end

function matchingPairs(adj list number, n number) returns list number
end

function isPerfectMatching(adj list number, n number) returns bool
end

function unmatchedCount(adj list number, n number) returns number
end

function matchingWeight(adj list number, n number) returns number
end

function vertexCoverSize(adj list number, n number) returns number
end

function isMatched(adj list number, n number, node number) returns bool
end
end
```